

Introduccion2

July 2, 2026

PAO25_25_02_Python_Datatype

June 22, 2026

01PAO26-26 - Python 1, Data Types

Kevin Xavier Suasnavas Villarreal

Link a Github

1 Comentarios

1.0.1 ¿Qué son?

Texto contenido en ficheros Python que es ignorado por el intérprete; es decir, no es ejecutado.

1.0.2 ¿Cuál es su utilidad?

Se puede utilizar para documentar código y hacerlo más legible.

Preferiblemente, trataremos de hacer código fácil de entender y que necesite pocos comentarios, en lugar de vernos forzados a recurrir a los comentarios para explicar el código.

1.0.3 Tipos de comentarios

Comentarios de una línea

Texto precedido por #.

Se suele usar para documentar expresiones sencillas.

```
[2]: # Esto es una instrucción print  
  
print("Hello world") # Esto es una instrucción print
```

Hello world

Comentarios de varias líneas

Texto encapsulado en triples comillas (que pueden ser tanto comillas simples como dobles).

Se suele usar para documentar bloques de código más significativos.

```
[4]: def producto(x, y):  
    '''  
    Esta función recibe dos números como parámetros y devuelve  
    como resultado el producto de los mismos.  
    '''  
    return x * y
```

2 Literales, variables y tipos de datos básicos

De forma muy genérica, al ejecutarse un programa Python, simplemente se realizan operaciones sobre objetos.

Estos dos términos son fundamentales.

Objetos: cualquier tipo de datos (números, caracteres o datos más complejos).

Operaciones: cómo manipulamos estos datos.

Ejemplo:

```
[5]: 4 + 3
```

```
[5]: 7
```

2.1 Literales

Python tiene una serie de tipos de datos integrados en el propio lenguaje.

Los literales son expresiones que generan objetos de estos tipos.

Estos objetos, según su tipo, pueden ser:

- Simples o compuestos.
- Mutables o inmutables.

Literales simples

Enteros

Decimales o punto flotante

Booleano

```
[6]: # print(4)           # número entero  
     # print(4.2)        # número en coma flotante  
     # print("Hello world!") # string  
     # print(False)
```

Literales compuestos

Tuplas

Listas

Diccionarios

Conjuntos

```
[7]: # print([1, 2, 3, 3])                # Lista - mutable

# print({'Nombre': 'John Doe', 'edad': 30}) # Diccionario - mutable

# print({1, 2, 3, 3})                    # Conjunto - mutable

# print((4, 5))                          # Tupla - immutable

# 2, 4                                    # Tupla
```

2.2 Variables

Referencias a objetos.

Las variables y los objetos se almacenan en diferentes zonas de memoria.

Las variables siempre referencian a objetos y nunca a otras variables.

Objetos sí que pueden referenciar a otros objetos. Ejemplo: listas.

Sentencia de asignación:

<nombre_variable> = <objeto>

```
[8]: # Asignación de variables

# a = 5

# print(a)
```

```
[9]: # a = 1                # entero
# b = 4.0                 # coma flotante
# c = "ITQ"               # string
# d = 10 + 1j              # número complejo
# e = True                 # boolean
# f = None                 # None

# visualizar valor de las variables y su tipo

# print(a)
# print(type(a))

# print(b)
# print(type(b))

# print(c)
# print(type(c))

# print(d)
# print(type(d))
```

```
# print(e)
# print(type(e))

# print(f)
# print(type(f))
```

Las variables no tienen tipo.

Las variables apuntan a objetos que sí lo tienen.

Dado que Python es un lenguaje de tipado dinámico, la misma variable puede apuntar, en momentos diferentes de la ejecución del programa, a objetos de diferente tipo.

```
[10]: # a = 3

# print(a)
# print(type(a))

# a = "Pablo García"

# print(a)
# print(type(a))

# a = 4.5

# print(a)
# print(type(a))
```

Garbage collection: Cuando un objeto deja de estar referenciado, se elimina automáticamente.

Identificadores

Podemos obtener un identificador único para los objetos referenciados por variables.

Este identificador se obtiene a partir de la dirección de memoria.

```
[11]: a = 3
print(id(a))

a = "Pablo García"
print(id(a))

a = 4.5
print(id(a))
```

```
140714462298984
2965582460080
2965581854192
```

Referencias compartidas: un mismo objeto puede ser referenciado por más de una variable.

Variables que referencian al mismo objeto tienen el mismo identificador.

```
[12]: a = 4567
```

```
print(id(a))
```

```
2965581854832
```

```
[13]: b = a
```

```
print(id(b))
```

```
2965581854832
```

```
[14]: c = 4567
```

```
print(id(c))
```

```
2965581855248
```

```
[15]: a = 25
```

```
b = 25
```

```
print(id(a))
```

```
print(id(b))
```

```
print(id(25))
```

```
140714462299688
```

```
140714462299688
```

```
140714462299688
```

```
[16]: # Ojo con los enteros "grandes" [-5, 256]
```

```
a = 258
```

```
b = 258
```

```
print(id(a))
```

```
print(id(b))
```

```
print(id(258))
```

```
2965581856176
```

```
2965581854928
```

```
2965581855216
```

Referencia al mismo objeto a través de asignar una variable a otra.

```
[17]: a = 400
```

```
b = a
```

```
print(id(a))
```

```
print(id(b))
```

2965581854640
2965581854640

Las variables pueden aparecer en expresiones.

```
[18]: a = 3  
      b = 5  
  
      print(a + b)
```

8

```
[19]: c = a + b  
  
      print(c)  
      print(id(c))
```

8

140714462299144

Respecto a los nombres de las variables...

No se puede poner números delante del nombre de las variables.

Por convención, evitar CamelCase. Mejor usar snake_case: uso de “_” para separar palabras.

El lenguaje diferencia entre mayúsculas y minúsculas.

Deben ser descriptivos.

Hay palabras o métodos reservados (Built-ins y Keywords).

¡Ojo con reasignar un nombre reservado!

```
[20]: print(pow(3, 2))
```

9

```
[21]: print(pow(3, 2))  
  
      pow = 1      # built-in reasignado  
  
      print(pow)  
  
      print(pow(3, 2))
```

9

1

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[21], line 7  
      3 pow = 1      # built-in reasignado  
      4
```

```
5 print(pow)
6
----> 7 print(pow(3, 2))
```

`TypeError: 'int' object is not callable`

```
[23]: def pow(a, b):
      return a + b
```

Asignación múltiple de variables

```
[24]: x, y, z = 1, 2, 3

      print(x, y, z)

      t = x, y, z, 7, "Python"

      print(t)

      print(type(t))
```

```
1 2 3
(1, 2, 3, 7, 'Python')
<class 'tuple'>
```

Esta técnica tiene un uso interesante: el intercambio de valores entre dos variables.

```
[25]: a = 1
      b = 2

      a, b = b, a

      print(a, b)
```

```
2 1
```

```
[26]: a = 1
      b = 2

      c = a

      a = b

      b = c

      print(a, b, c)
```

```
2 1 1
```

2.3 Tipos de datos básicos

Bool

2 posibles valores: True o False.

```
[27]: a = False
      b = True

      print(a)
      print(type(a))

      print(b)
      print(type(b))
```

False

<class 'bool'>

True

<class 'bool'>

True y False también son objetos que se guardan en caché, al igual que los enteros pequeños.

```
[28]: a = True
      b = True

      print(id(a))
      print(id(b))

      print(a is b)

      print(a == b)
```

140714460822080

140714460822080

True

True

Números

```
[29]: print(2)           # Enteros, sin parte fraccional.

      print(3.4)         # Números en coma flotante, con parte fraccional.

      print(2 + 4j)       # Números complejos.

      print(1 / 2)        # Números racionales.
```

2

3.4

(2+4j)

0.5

Diferentes representaciones: base 10, 2, 8 y 16.


```
[30]: x = 58          # Decimal
      z = 0b00111010 # Binario
      w = 0o72        # Octal
      y = 0x3A        # Hexadecimal
      print(x == y == z == w)
```

True

Strings

Cadenas de caracteres.

Son secuencias: la posición de los caracteres es importante.

Son inmutables: las operaciones sobre strings no cambian el string original.

```
[31]: s = 'John "ee" Doe'
      print(s[0])      # Primer carácter.
      print(s[-1])     # Último carácter.
      print(s[1:8:2])  # Slicing.
      print(s[:])      # Todo el string.
      print(s + "e")   # Concatenación.
```

J

e

on"e

John "ee" Doe

John "ee" Doe

2.4 Conversión entre tipos

A veces queremos que un objeto sea de un tipo específico.

Podemos obtener objetos de un tipo a partir de objetos de un tipo diferente (casting).

```
[32]: a = int(2.8)      # a será 2
      b = int("3")      # b será 3

      c = float(1)      # c será 1.0
      d = float("3")    # d será 3.0

      e = str(2)        # e será "2"
      f = str(3.0)      # f será "3.0"
```

```

g = bool("a")           # g será True
h = bool("")           # h será False

i = bool(3)            # i será True
j = bool(0)            # j será False

k = bool(None)

print(a)
print(type(a))

print(b)
print(type(b))

print(c)
print(type(c))

print(d)
print(type(d))

print(e)
print(type(e))

print(f)
print(type(f))

print(g)
print(type(g))

print(h)
print(type(h))

print(i)
print(type(i))

print(j)
print(type(j))

print(k)

```

```

2
<class 'int'>
3
<class 'int'>
1.0
<class 'float'>
3.0

```

```
<class 'float'>
2
<class 'str'>
3.0
<class 'str'>
True
<class 'bool'>
False
<class 'bool'>
True
<class 'bool'>
False
<class 'bool'>
False
```

```
[33]: print(7 / 4)          # División convencional. Resultado de tipo float.

      print(7 // 4)       # División entera. Resultado de tipo int.

      print(int(7 / 4))   # Conversión del resultado de float a int.
```

```
1.75
```

```
1
```

```
1
```

2.5 Operadores

Python 3: precedencia en operaciones

Combinación de valores, variables y operadores.

Operadores y operandos.

Operadores aritméticos

Operador

Descripción

a + b

Suma

a - b

Resta

a / b

División

a // b

División entera

a % b

Módulo / Resto

`a * b`

Multipliación

`a ** b`

Exponenciación

```
[34]: x = 3
      y = 2

      print("x + y =", x + y)
      print("x - y =", x - y)
      print("x * y =", x * y)
      print("x / y =", x / y)
      print("x // y =", x // y)
      print("x % y =", x % y)
      print("x ** y =", x ** y)
```

`x + y = 5`

`x - y = 1`

`x * y = 6`

`x / y = 1.5`

`x // y = 1`

`x % y = 1`

`x ** y = 9`

Operadores de comparación

Operador

Descripción

`a > b`

Mayor

`a < b`

Menor

`a == b`

Igualdad

`a != b`

Desigualdad

`a >= b`

Mayor o igual

`a <= b`

Menor o igual

```
[35]: x = 10
      y = 12

      print("x > y es", x > y)
      print("x < y es", x < y)
      print("x == y es", x == y)
      print("x != y es", x != y)
      print("x >= y es", x >= y)
      print("x <= y es", x <= y)
```

```
x > y es False
x < y es True
x == y es False
x != y es True
x >= y es False
x <= y es True
```

Operadores lógicos

Operador

Descripción

a and b

True, si ambos son True.

a or b

True, si alguno de los dos es True.

$a \wedge b$

XOR. True, si solo uno de los dos es True.

not a

Negación.

Enlace a Tablas de Verdad

```
[36]: x = True
      y = False

      print("x and y es :", x and y)
      print("x or y es :", x or y)
      print("x xor y es :", x ^ y)
      print("not x es :", not x)
```

```
x and y es : False
x or y es : True
x xor y es : True
not x es : False
```

Operadores Bitwise / Binarios

Operador

Descripción

a & b

And binario

a | b

Or binario

a ^ b

Xor binario

~a

Not binario

a » b

Desplazamiento binario a la derecha

a « b

Desplazamiento binario a la izquierda

```
[38]: # x = 0b01100110
      # y = 0b00110011

      # print("Not x = " + bin(~x))

      # print("x and y = " + bin(x & y))

      # print("x or y = " + bin(x | y))

      # print("x xor y = " + bin(x ^ y))

      # print("x << 2 = " + bin(x << 2))

      # print("x >> 2 = " + bin(x >> 2))
```

Operadores de asignación

Operador

Descripción

=

Asignación

+=

Suma y asignación

`-=`

Resta y asignación

`*=`

Multiplicación y asignación

`/=`

División y asignación

`%=`

Módulo y asignación

`//=`

División entera y asignación

`**=`

Exponenciación y asignación

`&=`

And y asignación

`|=`

Or y asignación

`^=`

Xor y asignación

`»=`

Desplazamiento a la derecha y asignación

`«=`

Desplazamiento a la izquierda y asignación

```
[39]: a = 5

a *= 3      # a = a * 3

a += 1      # No existe a++, ni ++a, a--, --a

print(a)

b = 6

b -= 2      # b = b - 2

print(b)
```

16

4

Operadores de identidad

Operador

Descripción

a is b

True, si ambos operandos son una referencia al mismo objeto.

a is not b

True, si ambos operandos no son una referencia al mismo objeto.

```
[40]: a = 4444
      b = a

      print(a is b)
      print(a is not b)
```

True

False

Operadores de pertenencia

Operador

Descripción

a in b

True, si a se encuentra en la secuencia b.

a not in b

True, si a no se encuentra en la secuencia b.

```
[41]: x = "Hola Mundo"
      y = {1: "a", 2: "b"}

      print("H" in x)           # True
      print("hola" not in x)    # True
      print(1 in y)             # True
      print("a" in y)           # False
```

True

True

True

False

2.6 Entrada de valores

```
[42]: valor = input("Inserte valor: ")  
  
print(valor)  
  
print(type(valor))
```

```
Inserte valor: true  
  
true  
<class 'str'>
```

```
[43]: grados_c = int(input("Conversión de grados a Fahrenheit, inserte un valor: "))  
  
print(f"Grados F: {1.8 * grados_c + 32}")
```

```
Conversión de grados a Fahrenheit, inserte un valor: 8
```

```
Grados F: 46.4
```

3 Tipos de datos compuestos (colecciones)

3.1 Listas

Una colección de objetos.

Mutables.

Tipos arbitrarios heterogéneos.

Puede contener duplicados.

No tienen tamaño fijo. Pueden contener tantos elementos como quepan en memoria.

Los elementos van ordenados por posición.

Se acceden usando la sintaxis: [index].

Los índices van de 0 a n-1, donde n es el número de elementos de la lista.

Son un tipo de Secuencia, al igual que los strings; por lo tanto, el orden (es decir, la posición de los objetos de la lista) es importante.

Soportan anidamiento.

Son una implementación del tipo abstracto de datos: Array Dinámico.

Operaciones con listas

Creación de listas.

```
[45]: letras = ['a', 'b', 'c', 'd']  
  
palabras = "Hola mundo como estas".split()  
  
numeros = list(range(7))
```

```
print(letras)

print(palabras)

print(numeros)

print(type(numeros))
```

```
['a', 'b', 'c', 'd']
['Hola', 'mundo', 'como', 'estas']
[0, 1, 2, 3, 4, 5, 6]
<class 'list'>
```

[46]: *# Pueden contener elementos arbitrarios / heterogéneos*

```
mezcla = [1, 3.4, "a", None, False]

print(mezcla)

print(len(mezcla))    # Número de elementos de la lista
```

```
[1, 3.4, 'a', None, False]
5
```

[47]: *# Pueden incluso contener objetos más "complejos"*

```
lista_con_funcion = [1, 2, len, pow]

print(lista_con_funcion)
```

```
[1, 2, <built-in function len>, <function pow at 0x000002B27A8F0B80>]
```

[48]: *# Pueden contener duplicados*

```
lista_con_duplicados = [1, 2, 3, 3, 3, 4]

print(lista_con_duplicados)
```

```
[1, 2, 3, 3, 3, 4]
```

Obtención de la longitud de una lista.

[49]: `letras = ["a", "b", "c", "d", 1]`

```
print(len(letras))
```

```
5
```

Acceso a un elemento de una lista.

```
[51]: print(letras[2])

      print(letras[-5])

      print(letras[0])
```

```
c
a
a
```

Slicing: obtención de un fragmento de una lista. Devuelve una copia de una parte de la lista.

Sintaxis: lista[inicio:fin:paso]

```
[52]: letras = ["a", "b", "c", "d", "e"]

      print(letras[1:3])

      print(letras[:3])

      print(letras[:-1])

      print(letras[2:])

      print(letras[:])

      print(letras[::2])
```

```
['b', 'c']
['a', 'b', 'c']
['a', 'b', 'c', 'd']
['c', 'd', 'e']
['a', 'b', 'c', 'd', 'e']
['a', 'c', 'e']
```

```
[53]: letras = ["a", "b", "c", "d"]

      print(letras)

      print(id(letras))

      a = letras[:]

      print(a)

      print(id(a))

      print(letras.copy())
```

```
print(id(letras.copy()))
```

```
['a', 'b', 'c', 'd']  
2965582661824  
['a', 'b', 'c', 'd']  
2965582805696  
['a', 'b', 'c', 'd']  
2965582988608
```

Añadir un elemento al final de la lista.

```
[54]: letras.append("e")
```

```
print(letras)  
  
print(id(letras))
```

```
['a', 'b', 'c', 'd', 'e']  
2965582661824
```

```
[55]: letras += "e"
```

```
print(letras)  
  
print(id(letras))
```

```
['a', 'b', 'c', 'd', 'e', 'e']  
2965582661824
```

Insertar en una posición.

```
[56]: print(len(letras))  
  
letras.insert(1, "g")  
  
print(len(letras))  
  
print(letras)  
  
print(id(letras))
```

```
6  
7  
['a', 'g', 'b', 'c', 'd', 'e', 'e']  
2965582661824
```

Modificación de la lista (individual).

```
[57]: letras[5] = "f"  
  
print(letras)
```

```
print(id(letras))
```

```
['a', 'g', 'b', 'c', 'd', 'f', 'e']  
2965582661824
```

```
[58]: # El índice debe estar dentro del rango de la lista
```

```
letras[20] = "r"
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[58], line 3  
      1 # El índice debe estar dentro del rango de la lista  
      2  
----> 3 letras[20] = "r"  
  
IndexError: list assignment index out of range
```

Modificación múltiple usando slicing.

```
[59]: letras = ["a", "b", "c", "f", "g", "h", "i", "j", "k"]
```

```
print(id(letras))
```

```
2965582665088
```

```
[60]: letras[0:7:2] = ["z", "x", "y", "p"]
```

```
print(letras)
```

```
# print(id(letras))
```

```
['z', 'b', 'x', 'f', 'y', 'h', 'p', 'j', 'k']
```

```
[61]: # Ojo con la diferencia entre modificación individual y múltiple.  
# La asignación individual de una lista crea anidamiento.
```

```
numeros = [1, 2, 3]
```

```
numeros[1] = [10, 20, 30]
```

```
print(numeros)
```

```
print(numeros[1][0])
```

```
numeros[1][2] = [100, 200]
```

```
print(numeros)

print(numeros[1][2][1])
```

```
[1, [10, 20, 30], 3]
10
[1, [10, 20, [100, 200]], 3]
200
```

Eliminar un elemento.

```
[62]: # letras.remove("f")

if "p" in letras:
    letras.remove("p")

# letras.remove("z")

print(letras)
```

```
['z', 'b', 'x', 'f', 'y', 'h', 'j', 'k']
```

```
[63]: # Elimina el elemento de la última posición y lo devuelve

elemento = letras.pop()

print(elemento)

print(letras)
```

```
k
['z', 'b', 'x', 'f', 'y', 'h', 'j']
```

```
[64]: numeros = [1, 2, 3]

print(numeros)

numeros[2] = [10, 20, 30]
```

```
[1, 2, 3]
```

```
[65]: print(numeros)

n = numeros[2].pop()

print(numeros)

print(n)
```

```
[1, 2, [10, 20, 30]]
```

```
[1, 2, [10, 20]]  
30
```

```
[66]: # numeros1 = 10  
  
# print(numeros1)  
  
# print(numeros)
```

```
[67]: lista = []  
  
a = lista.pop()
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[67], line 3  
      1 lista = []  
      2  
----> 3 a = lista.pop()  
  
IndexError: pop from empty list
```

Encontrar el índice de un elemento.

```
[68]: letras = ["a", "b", "c", "c"]  
  
if "a" in letras:  
    print(letras.index("a"))  
  
print(letras)
```

```
0  
['a', 'b', 'c', 'c']
```

Concatenar listas.

```
[69]: lacteos = ["queso", "leche"]  
  
frutas = ["naranja", "manzana"]  
  
print(id(lacteos))  
  
print(id(frutas))  
  
compra = lacteos + frutas  
  
print(id(compra))
```

```
print(compra)
```

2965582965504

2965582954880

2965583957440

['queso', 'leche', 'naranja', 'manzana']

[70]: *# Concatenación sin crear una nueva lista*

```
frutas = ["naranja", "manzana"]
```

```
print(id(frutas))
```

```
frutas.extend(["pera", "uvas"])
```

```
print(frutas)
```

```
print(id(frutas))
```

2965582839744

['naranja', 'manzana', 'pera', 'uvas']

2965582839744

[71]: *# Anidar sin crear una nueva lista*

```
frutas = ["naranja", "manzana"]
```

```
print(id(frutas))
```

```
frutas.append(["pera", "uvas"])
```

```
print(frutas)
```

```
print(id(frutas))
```

2965582954880

['naranja', 'manzana', ['pera', 'uvas']]

2965582954880

Replicar una lista.

[72]: `lacteos = ["queso", "leche"]`

```
print(lacteos * 3)
```

```
print(id(lacteos))
```

```
a = 3 * lacteos
```



```
print(a)

print(id(a))
```

```
['queso', 'leche', 'queso', 'leche', 'queso', 'leche']
2965582990976
['queso', 'leche', 'queso', 'leche', 'queso', 'leche']
2965583735104
```

Copiar una lista.

```
[73]: frutas2 = frutas.copy()

frutas2 = frutas[:]

print(frutas2)

print("id frutas = " + str(id(frutas)))

print("id frutas2 = " + str(id(frutas2)))
```

```
['naranja', 'manzana', ['pera', 'uvas']]
id frutas = 2965582954880
id frutas2 = 2965582961984
```

Ordenar una lista.

```
[74]: lista = [4, 3, 8, 1]

print(lista)

lista.sort()

print(lista)

lista.sort(reverse=True)

print(lista)
```

```
[4, 3, 8, 1]
[1, 3, 4, 8]
[8, 4, 3, 1]
```

```
[75]: # Los elementos deben ser comparables para poder ordenarse

lista = [1, "a"]

lista.sort()
```

```

TypeError                                Traceback (most recent call last)
Cell In[75], line 5
      1 # Los elementos deben ser comparables para poder ordenarse
      2
      3 lista = [1, "a"]
      4
----> 5 lista.sort()

TypeError: '<' not supported between instances of 'str' and 'int'

```

```

[76]: compra = ["Huevos", "Pan", "zapallo", "Leche", "Licor"]

print(sorted(compra))

print(compra)

```

```

['Huevos', 'Leche', 'Licor', 'Pan', 'zapallo']
['Huevos', 'Pan', 'zapallo', 'Leche', 'Licor']

```

Pertenencia.

```

[77]: lista = [1, 2, 3, 4]

print(1 in lista)

print(5 in lista)

```

```

True
False

```

Anidamiento.

```

[78]: letras = ["a", "b", "c", ["x", "y", ["i", "j", "k"]]]

print(letras[0])

print(letras[3][0])

print(letras[3][2][0])

```

```

a
x
i

```

```

[79]: print(letras[3])

```

```

['x', 'y', ['i', 'j', 'k']]

```

```
[80]: a = [1000, 2, 3]

b = a[:]          # También puede usarse a.copy()

print(id(a))

print(id(b))

print(id(a[0]))

print(id(b[0]))
```

```
2965582841792
2965582852160
2965583824720
2965583824720
```

Alias / Referencias en las listas

Las listas son mutables y las asignaciones modifican el mismo objeto.

Pasar listas como parámetros a funciones puede implicar riesgos si se modifican.

Cuando se desea trabajar con una copia independiente, es recomendable clonar o copiar la lista.

```
[81]: lista = [2, 4, 16, 32]

ref = lista

print(id(lista))

print(id(ref))

ref[2] = 64

print(ref)

print(lista)
```

```
2965557094784
2965557094784
[2, 4, 64, 32]
[2, 4, 64, 32]
```

```
[82]: lista = [2000, 4, 16, 32]

copia = lista[:]

copia = lista.copy()
```

```

copia[2] = 64

print(lista)

print(copia)

print(id(lista))

print(id(copia))

print(id(lista[2]))

print(id(copia[2]))

```

```

[2000, 4, 16, 32]
[2000, 4, 64, 32]
2965582738688
2965582906112
140714462299400
140714462300936

```

[83]: *# Las listas anidadas se copian por referencia (copia superficial)*

```

lista = [2, 4, 16, 32, [34, 10, [5, 5]]]

copia = lista.copy()

copia[3] = 20

copia[4][0] = 28

print(copia)

print(lista)

```

```

[2, 4, 16, 20, [28, 10, [5, 5]]]
[2, 4, 16, 32, [28, 10, [5, 5]]]

```

[84]: *# lista = [0, 1, [10, 20]]*

```

# print(id(lista))

# lista2 = [10, 20]

# lista = [0, 1, lista2]

# copia = lista[:]      # También puede usarse lista.copy()

```

```
# print(copia)

# print(id(copia))

# print(id(lista[2]))

# print(id(copia[2]))

# copia[2][0] = 40

# print(copia)

# print(lista)
```

[85]: *# Evitar que listas anidadas se copien por referencia*

```
# import copy

# lista = [2, 4, 16, 32, [34, 10, [5, 5]]]

# copia = copy.deepcopy(lista)

# copia[0] = 454

# copia[4][2][0] = 64

# print(lista)

# print(copia)

# print(f"{id(lista)} - {id(copia)}")

# print(f"{id(lista[4])} - {id(copia[4])}")
```

3.2 Diccionarios

Colección de parejas clave-valor.

Son mutables.

Claves:

Cualquier objeto inmutable.

Sólo pueden aparecer una vez en el diccionario.

Valores:

Sin restricciones. Cualquier objeto (enteros, strings, listas, etc.) puede hacer de valor.

Desde la versión 3.7 están ordenados.

Se pueden ver como listas indexadas por cualquier objeto inmutable, no necesariamente por números enteros.

A diferencia de las listas, no son secuencias. Son mappings.

Operaciones con diccionarios

Creación de diccionarios.

[86]: *# Creación simple usando una expresión literal.*

```
persona = {  
    "DNI": "11111111D",  
    "Nombre": "Carlos",  
    "Edad": 34  
}  
  
print(persona)
```

{'DNI': '11111111D', 'Nombre': 'Carlos', 'Edad': 34}

[87]: *# Creación uniendo dos colecciones.*

```
nombres = ["Pablo", "Manolo", "Pepe", "Juan"]  
  
edades = [52, 14, 65]  
  
datos = dict(zip(nombres, edades))  
  
print(datos)
```

{'Pablo': 52, 'Manolo': 14, 'Pepe': 65}

[88]: *# Creación pasando claves y valores a la función dict()*

```
persona2 = dict(  
    nombre="Rosa",  
    apellido="Garcia"  
)  
  
print(persona2)
```

{'nombre': 'Rosa', 'apellido': 'Garcia'}

[89]: *# Creación usando una lista de tuplas de dos elementos.*

```
persona2 = dict([  
    ("nombre", "Rosa"),  
    ("apellido", "Garcia")  
])
```

```
print(persona2)
```

```
{'nombre': 'Rosa', 'apellido': 'Garcia'}
```

```
[90]: # Creación incremental por medio de asignación
      # (como las claves no existen, se crean nuevos items)

      persona = {}

      print(persona["DNI"])

      persona["DNI"] = "11111111D"

      persona["Nombre"] = "Carlos"

      persona["Edad"] = 34

      persona["DNI"] = "222222222"

      print(persona)
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[90], line 6
      2 # (como las claves no existen, se crean nuevos items)
      3
      4 persona = {}
      5
----> 6 print(persona["DNI"])
      7
      8 persona["DNI"] = "11111111D"
      9

KeyError: 'DNI'
```

Acceso a un valor a través de la clave.

```
[91]: persona = {
      "DNI": "11111111D",
      "Nombre": "Carlos",
      "Edad": 34
      }

      print(persona["Nombre"])
```

Carlos

[92]: *# Acceso a claves inexistentes o por índice produce error*

```
# persona[1]  
  
# print(persona["Nombre"])  
  
# if "Trabajo" in persona:  
#     print(persona["Trabajo"])  
# else:  
#     print("En el paro")  
  
# persona.get("Trabajo", "En el paro")
```

Modificación de un valor a través de la clave.

[93]:

```
# persona = {  
#     "DNI": "11111111D",  
#     "Nombre": "Carlos",  
#     "Edad": 34  
# }  
  
# persona["Nombre"] = "Fernando"  
  
# persona["Edad"] += 1  
  
# print(persona)
```

Añadir un valor a través de la clave.

[94]:

```
# persona = {  
#     "DNI": "11111111D",  
#     "Nombre": "Carlos",  
#     "Edad": 34  
# }  
  
# persona["Ciudad"] = "Valencia"  
  
# print(persona)
```

Eliminación de un valor a través de la clave.

[95]:

```
# del persona["Ciudad"]  
  
# value = persona.pop("Edad")  
  
# print(persona)  
  
# print(value)
```


Comprobación de existencia de una clave.

```
[96]: # print("Nombre" in persona)

# print("Apellido" in persona)
```

Recuperación del valor de una clave, indicando un valor por defecto en caso de ausencia.

```
[97]: # persona = {"Nombre": "Carlos"}

# value = persona.get("Nombre")

# print(value)

# persona["Estatura"]

# value = persona.get("Estatura", 180)

# print(value)
```

Anidamiento.

```
[98]: # persona = {
#     "Trabajos": ["desarrollador", "gestor"],
#     "Direccion": {
#         "Calle": "Pintor Sorolla",
#         "Ciudad": "Valencia"
#     }
# }

# print(persona["Direccion"])

# print(persona["Direccion"]["Ciudad"])
```

Métodos items(), keys() y values().

```
[99]: # persona = {
#     "DNI": "11111111D",
#     "Nombre": "Carlos",
#     "Edad": 34
# }

# print(list(persona.items()))

# print(list(persona.keys()))

# print(list(persona.values()))
```

```
[100]: # dict_simple = {
#       "ID": "XCSDe1194",
#       "Nombre": "Carlos",
#       "Edad": 34
#     }

# Iterar sobre las claves

# print("Claves\n")

# for key in dict_simple.keys():      # o simplemente: for key in dict_simple:
#     print(key)

# print("\nValores\n")

# Iterar sobre los valores

# for value in dict_simple.values():
#     print(value)
```

```
[101]: # dict_simple = {
#       "ID": "XCSDe1194",
#       "Nombre": "Carlos",
#       "Edad": 34
#     }

# Iterar sobre clave y valor usando items()

# for key, value in dict_simple.items():
#     print("Clave: " + str(key) + ", valor: " + str(value))

# items() devuelve tuplas

# for item in dict_simple.items():
#     print("Item: " + str(item))

# Usar zip() para recorrer claves y valores

# for key, value in zip(dict_simple.keys(), dict_simple.values()):
#     print("Clave: " + str(key) + ", valor: " + str(value))
```

3.3 Sets (Conjuntos)

Al igual que las listas:

Colección de elementos.

Tipos arbitrarios.

Mutables.

No tienen tamaño fijo. Pueden contener tantos elementos como quepan en memoria.

A diferencia de las listas:

No pueden tener elementos duplicados.

Se definen por medio de llaves { }.

Los elementos no van ordenados por posición. No existe un orden establecido.

Solo pueden contener objetos inmutables.

No soportan anidamiento.

Operaciones con conjuntos

Creación de conjuntos.

```
[102]: # set1 = {0, 1, 1, 2, 3, 4, 4}

# print(set1)

# set2 = {"user1", 12, 2}

# print(set2)

# set3 = set(range(7))

# print(set3)

# set4 = set([0, 1, 2, 3, 4, 0, 1])

# print(set4)
```

```
[103]: # Observa la diferencia entre listas y conjuntos

# s = "aabbcc"

# print(list(s))

# print(set(s))
```

Acceso por índice genera error.

```
[104]: # set1 = {0, 1, 2}

# print(set1[0])
```

Unión, intersección y diferencia.

```
[105]: # set1 = {0, 1, 1, 2, 3, 4, 5, 8, 13, 21}

# set2 = set([0, 1, 2, 3, 4, 42])
```

```

# Unión
# print(set1 | set2)

# Intersección
# print(set1 & set2)

# Diferencia
# print(set1 - set2)

# print(set2 - set1)

```

```

[106]: # a = [1, 2]

# b = [2, 3]

# print(set(a) & set(b))

```

```

[107]: # Además de los operadores, que operan únicamente con Sets,
# también se pueden usar métodos que operan sobre cualquier
# objeto iterable.

# conjunto = {0, 1, 2}

# lista = [1, 3, 3]

# print(conjunto.union(lista))

# print(conjunto.intersection(lista))

# print(conjunto.difference(lista))

```

Comparación de conjuntos.

```

[108]: # set1 = {0, 1, 1, 2, 3, 4, 5, 8, 13, 21}

# set2 = set([0, 1, 2, 3, 4])

# print(set2.issubset(set1))

# print(set1.issuperset(set2))

# print(set1.isdisjoint(set2))

```

Pertenencia.

```

[109]: # words = {"calm", "balm"}

```

```
# print("calm" in words)
```

Anidamiento.

```
[110]: # Los conjuntos no soportan anidamiento, pero como permiten
# elementos inmutables, se pueden "anidar" tuplas.

# nested_set = {1, (1, 1, 1), 2, 3, (1, 1, 1)}

# print(nested_set)
```

Modificación de conjuntos.

```
[111]: # A través del operador de asignación

# set1 = {"a", "b", "c"}

# set2 = {"a", "d"}

# set1 /= set2          # set1 = set1 / set2

# set1 &= set2

# set1 -= set2

# print(set1)
```

```
[112]: # A través del método update()

# set1 = {"a", "b", "c"}

# set2 = {"a", "d"}

# set1.update(set2)

# set1.intersection_update(set2)

# set1.difference_update(set2)

# print(set1)
```

```
[113]: # A través de los métodos add() y remove()

# set1 = {"a", "b", "c"}

# set1.add("d")

# set1.remove("a")
```

```
# print(set1)
```

3.4 Tuplas

Al igual que las listas:

Colección de elementos.

Tipos arbitrarios.

Puede contener duplicados.

No tienen tamaño fijo. Pueden contener tantos elementos como quepan en memoria.

Los elementos van ordenados por posición.

Acceso mediante la sintaxis [index].

Los índices van desde 0 hasta n-1, donde n es el número de elementos de la tupla.

Son secuencias donde el orden de los elementos importa.

Soportan anidamiento.

A diferencia de las listas:

Se definen mediante paréntesis ().

Son inmutables.

¿Por qué tuplas?

Representación de una colección fija de elementos (por ejemplo, una fecha).

Pueden usarse en contextos que requieren inmutabilidad (por ejemplo, como claves de un diccionario).

Operaciones con tuplas

Creación de tuplas.

```
[114]: # tuple1 = ("Foo", 34, 5.0, 34)

# print(tuple1)

# tuple2 = 1, 2, 3

# print(tuple2)

# tuple3 = tuple(range(10))

# print(tuple3)

# tuple4 = tuple([0, 1, 2, 3, 4])
```

```
# print(tuple4)
```

```
[115]: # Ojo con las tuplas de un elemento.  
# Los paréntesis se interpretan como  
# indicadores de precedencia de operadores.  
  
# singleton_number = (1)  
  
# type(singleton_number)
```

```
[116]: # Creación de una tupla de un elemento.  
  
# singleton_tuple = (1,)  
  
# print(type(singleton_tuple))  
  
# print(singleton_tuple)
```

Obtención del número de elementos.

```
[117]: # print(len(tuple1))
```

Acceso por índice.

```
[118]: # tuple1 = ("Foo", 1, 2, 3)  
  
# print(tuple1[0])           # Primer elemento  
  
# print(tuple1[len(tuple1)-1]) # Último elemento  
  
# print(tuple1[-1])          # Índices negativos comienzan desde el final  
  
# print(tuple1[-len(tuple1)]) # Primer elemento
```

La asignación a tuplas falla porque son inmutables.

```
[119]: # tuple1[0] = "bar"
```

```
[120]: # print(id(tuple1))  
  
# lista = list(tuple1)  
  
# lista[0] = "bar"  
  
# tuple1 = tuple(lista)  
  
# print(id(tuple1))  
  
# print(tuple1)
```

Contar el número de ocurrencias de un elemento.

```
[121]: # tuple1 = ("Foo", 34, 5.0, 34)

# print(tuple1.count(34))
```

Encontrar el índice de un elemento.

```
[122]: # tuple1 = ("Foo", 34, 5.0, 34)

# indice = tuple1.index(34)

# print(indice)

# print(tuple1[indice])
```

```
[123]: # Si el elemento no existe, se produce un error.

# tuple1 = ("Foo", 34, 5.0, 34)

# print(tuple1.index(1))
```

```
[124]: # Comprobar si existe antes de buscarlo.

# tuple1 = ("Foo", 34, 5.0, 34)

# elemento = 35

# if elemento in tuple1:
#     print(tuple1.index(elemento))
# else:
#     print(str(elemento) + " not found")
```

Desempaquetar una tupla.

```
[125]: # tuple1 = (1, 2, 3, 4)

# a, b, c, d = tuple1      # a,b,c - a,b,c,d - a,b,_,_ - a,*_

# print(a)

# print(b)

# print(_)

# print(d)
```



```
[126]: # a, b, *resto = tuple1

# print(a)

# print(b)

# print(resto)
```

3.5 Secuencias

Formalmente, objetos iterables no materializados.

No son listas. Utiliza list() para materializarlas.

Ejemplos: range o enumerate.

```
[127]: # list(range(0, 10, 2))
```

```
[128]: # lista = [10, 20, 30]

# print(list(enumerate(lista)))
```

```
[129]: # enumerate para iterar una colección (índice y valor)

# lista = [10, 20, 30]

# for index, value in enumerate(lista):
#     print("Index = " + str(index) + ". Value = " + str(value))

# for index, value in [(0, 10), (1, 20), (2, 30)]:
#     print("Index = " + str(index) + ". Value = " + str(value))
```

```
[130]: # enumerate para iterar una colección (índice y valor)

# for index, value in enumerate(range(0, 10, 2)):
#     print("Index = " + str(index) + ". Value = " + str(value))

# for value in enumerate(range(0, 10, 2)):
#     print(value)
```

```
[131]: # zip para unir, elemento a elemento, dos colecciones,
# retornando una lista de tuplas.
# Útil para iterar dos listas al mismo tiempo.

# nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34, 45]

# for nombre, edad in zip(nombres, edades):
```

```
#     print("Nombre: " + nombre + ", edad: " + str(edad))
```

```
[132]: # nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34]

# for i in range(0, len(nombres)):
#     print(f"Nombre es {nombres[i]} y edad es {edades[i]}")
```

```
[133]: # nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34]

# jugadores = zip(nombres, edades)    # Genera una secuencia

# print(list(jugadores))

# print(type(jugadores))
```

```
[134]: # Listas de diferentes longitudes.

# nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34, 44, 33]

# jugadores = zip(nombres, edades)

# print(list(jugadores))
```

```
[135]: # unzip

# nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34]

# alturas = [120, 130, 140]

# jugadores = zip(nombres, edades, alturas)

# print(list(jugadores))

# ns, es, al = zip(*jugadores)

# print(list(ns))

# print(es)
```

```
# print(al)
```

```
[136]: # nombres = ["Manolo", "Pepe", "Luis"]

# edades = [31, 34, 34]

# dd = [23, 34, 45]

# jugadores_z = zip(nombres, edades, dd)

# print(list(jugadores_z))

# jugadores_uz = zip(*jugadores_z)

# print(list(jugadores_uz))
```

3.6 Para terminar, volvemos a enfatizar un matiz importante...

En Python todo son objetos.

Cada objeto tiene:

Identidad: nunca cambia una vez creado. Es comparable con una dirección de memoria. El operador `is` compara la identidad y la función `id()` devuelve dicha identidad.

Tipo: determina los posibles valores y operaciones que puede realizar el objeto. La función `type()` devuelve el tipo y este no cambia.

Valor: puede corresponder a objetos mutables o inmutables.

Tipos mutables: `list`, `dict`, `set` y los tipos definidos por el usuario.

Tipos inmutables: `int`, `float`, `bool`, `str` y `tuple`.

```
[137]: # Asignación

# list_numbers = [1, 2, 3]      # Lista (mutable)

# tuple_numbers = (1, 2, 3)     # Tupla (inmutable)

# print(list_numbers[0])

# print(tuple_numbers[0])

# list_numbers[0] = 100

# tuple_numbers[0] = 100

# print(list_numbers)

# print(tuple_numbers)
```

```
# tuple_l_numbers = list(tuple_numbers)

# tuple_l_numbers[0] = 100

# tuple_numbers = tuple(tuple_l_numbers)

# print(tuple_numbers)
```

```
[138]: # Identidad

# list_numbers = [1, 2, 3]

# tuple_numbers = (1, 2, 3)

# print("Id list_numbers: " + str(id(list_numbers)))

# print("Id tuple_numbers: " + str(id(tuple_numbers)))

# list_numbers += [4, 5, 6]      # La lista original se extiende

# tuple_numbers += (4, 5, 6)    # Se crea un nuevo objeto

# print(list_numbers)

# print(tuple_numbers)

# print("Id list_numbers: " + str(id(list_numbers)))

# print("Id tuple_numbers: " + str(id(tuple_numbers)))
```

```
[139]: # Referencias

# list_numbers = [1, 2, 3]

# list_numbers_2 = list_numbers    # list_numbers_2 referencia a list_numbers

# print("Id list_numbers: " + str(id(list_numbers)))

# print("Id list_numbers_2: " + str(id(list_numbers_2)))

# list_numbers.append(4)           # Se actualiza list_numbers_2 también

# print(list_numbers)

# print(list_numbers_2)
```

```
# print("Id list_numbers: " + str(id(list_numbers)))

# print("Id list_numbers_2: " + str(id(list_numbers_2)))
```

```
[140]: # text = "Hola"      # Inmutable

# text_2 = text      # Referencia

# print("Id text: " + str(id(text)))

# print("Id text_2: " + str(id(text_2)))

# text += " Mundo"

# print(text)

# print(text_2)

# print("Id text: " + str(id(text)))

# print("Id text_2: " + str(id(text_2)))
```

```
[141]: # teams = ["Team A", "Team B", "Team C"]      # Mutable

# player = (23, teams)                          # Tupla (immutable)

# print(type(player))

# print(player)

# print(id(player))

# teams[2] = "Team J"

# print(player)

# print(id(player))

# player[1][0] = "Team XX"

# print(teams)
```

3.7 Ejercicios

Escribe un programa que muestre por pantalla la concatenación de un número y una cadena de caracteres. Para obtener esta concatenación puedes usar uno de los operadores explicados en este tema.

Ejemplo: dado el número 3 y la cadena 'abc', el programa mostrará la cadena '3abc'.

Escribe un programa que muestre por pantalla un valor booleano que indique si un número entero N está contenido en un intervalo semiabierto $[a,b)$, el cual establece una cota inferior a (inclusive) y una cota superior b (exclusive) para N .

Escribe un programa que, dados dos strings $S1$ y $S2$ y dos números enteros $N1$ y $N2$, determine si el substring que en $S1$ se extiende desde la posición $N1$ hasta la $N2$ (ambos inclusive) está contenido en $S2$.

Dada una lista con elementos duplicados, escribir un programa que muestre una nueva lista con el mismo contenido que la primera, pero sin elementos duplicados.

Escribe un programa que, dada una lista de strings L , un string s perteneciente a L y un string t , reemplace s por t en L . El programa debe mostrar la lista resultante por pantalla.

Escribe un programa que defina una tupla con elementos numéricos, reemplace el valor del último por un valor diferente y muestre la tupla por pantalla. Recuerda que las tuplas son inmutables. Tendrás que usar objetos intermedios.

Dada la lista $[1,2,3,4,5,6,7,8]$, escribe un programa que, a partir de esta lista, obtenga la lista $[8,6,4,2]$ y la muestre por pantalla.

Escribe un programa que, dada una tupla y un índice válido i , elimine el elemento de la tupla que se encuentra en la posición i . Para este ejercicio solo puedes usar objetos de tipo tupla. No puedes convertir la tupla a una lista, por ejemplo.

Escribe un programa que obtenga la mediana de una lista de números. Recuerda que la mediana M de una lista de números L es el número que cumple la siguiente propiedad: la mitad de los números de L son superiores a M y la otra mitad son inferiores. Cuando el número de elementos de L es par, se puede considerar que hay dos medianas. No obstante, en este ejercicio consideraremos que únicamente existe una mediana.

4 Soluciones

[142]: *# Ejercicio 1*

```
x = 3
y = "abc"

print(str(x) + y)
```

3abc

[143]: *# Ejercicio 2*

```
N = 5
a = 0
b = 6

# print(a <= N)
# print(N < b)

print(a <= N and N < b)
```

True

```
[144]: # Ejercicio 3

S1 = "Java"
S2 = "JavaScript"

N1 = 0
N2 = 3

print(S1[N1:N2 + 1] in S2)
```

True

```
[145]: # Ejercicio 4

L = [1, 2, 2, 3, 3, 2, 3, 1, 4, 5, 4, 5, 5]

print(list(set(L)))
```

[1, 2, 3, 4, 5]

```
[146]: # Ejercicio 5

# Escribe un programa que, dada una lista de strings L,
# un string s perteneciente a L y un string t,
# reemplace s por t en L.
# El programa debe mostrar la lista resultante por pantalla.

L = ["Python", "Java", "C++", "Kotlin"]

s = "Java"
t = "JavaScript"

indice = L.index(s)

L[indice] = t

print(L)
```

['Python', 'JavaScript', 'C++', 'Kotlin']

```
[148]: # # Ejercicio 6
# #Escribe un programa que defina una tupla con elementos numéricos, reemplace
# el
# valor del último por un valor diferente y muestre
# la tupla por pantalla. Recuerda que las tuplas son inmutables. Tendrás que
# usar objetos intermedios.
tupla = (1,2,3,4)
```

```

lista = list(tupla)
lista[-1] = 5
print(tuple(lista))

```

(1, 2, 3, 5)

```

[149]: # # Ejercicio 7
# #Dada la lista [1,2,3,4,5,6,7,8] escribe un programa que, a partir de esta
# lista,
# #obtenga la lista [8,6,4,2] y la muestre por pantalla.
lista = [1,2,3,4,5,6,7,8]
print(lista[-1::-2]) #(inicio, fin, paso)

```

[8, 6, 4, 2]

```

[150]: # # Ejercicio 8
# #Escribe un programa que, dada una tupla y un índice válido i, elimine el
# elemento de la tupla que se encuentra en la posición i.
# #Para este ejercicio sólo puedes usar objetos de tipo tupla. No puedes
#   ↪ convertir
# la tupla a una lista, por ejemplo.
tupla = ('a', 'b', 'c', 'd', 'e', 'f')
i = 3
print(tupla[:i] + tupla[i+1:])

```

('a', 'b', 'c', 'e', 'f')

```

[151]: # # Ejercicio 9
# #Escribe un programa que obtenga la mediana de una lista de números. Recuerda
# que la mediana M de una lista de números L
# #es el número que cumple la siguiente propiedad: la mitad de los números de L
# son superiores a M y la otra mitad son inferiores.
# #Cuando el número de elementos de L es
# #par, se puede considerar que hay dos medianas. No obstante, en este
#   ↪ ejercicio
# consideraremos que únicamente existe una mediana.
L = [21, 9, 1, 5.5, 19, 3, 15]
L.sort()
print(L)
mediana = L[len(L)//2]
print(mediana)

```

[1, 3, 5.5, 9, 15, 19, 21]

9

[]: